

Generador y Analizador Estadístico

Azuara Yáñez Cecil Quimey
Universidad de Guanajuato
División de Ingenierías
Campus Irapuato - Salamanca
cq.azuarayanez@ugto.mx

León Amaya Juan Pablo
Universidad de Guanajuato
División de Ingenierías
Campus Irapuato - Salamanca
jp.leonamaya@ugto.mx

Rojas Mata Diego Moises
Universidad de Guanajuato
División de Ingenierías
Campus Irapuato - Salamanca
dm.rojasmata@ugto.mx

Resumen—Este reporte explica el diseño e implementación de un programa en lenguaje C creado para generar números aleatorios, ordenarlos y calcular sus medidas estadísticas. El programa cuenta con un menú interactivo para el usuario, pero también se puede ejecutar desde la terminal (línea de comandos) ingresando una semilla, lo que nos permite repetir las pruebas y obtener siempre los mismos resultados.

El programa puede generar los datos usando tres distribuciones: Uniforme, Normal (con el método de Box-Muller) y Laplace. Para asegurar que los números no se salgan del rango que pide el usuario, usamos la técnica de recorte (clipping), forzando a los números extremos a igualarse a los límites permitidos. Después de generar los datos, se hace una copia en la memoria para probar el tiempo que tardan 8 algoritmos de ordenamiento diferentes. Finalmente, el programa calcula 15 medidas estadísticas distintas. Los resultados de nuestras pruebas muestran cómo cambia el tiempo de ejecución dependiendo de la cantidad de datos y del algoritmo elegido.

I. INTRODUCCIÓN

El objetivo de esta práctica fue desarrollar un programa en C capaz de generar y analizar arreglos con grandes cantidades de datos (hasta 100,000). El sistema permite que el usuario capture datos manualmente o que la computadora los genere sola usando probabilidad Uniforme, Normal o Laplace. Además, el programa incluye 8 algoritmos de ordenamiento para medir y comparar cuánto tiempo tarda cada uno en acomodar los arreglos, lo que nos permite hacer un análisis real de su complejidad temporal y espacial.

II. METODOLOGÍA

Para que el código fuera más fácil de leer y de corregir, decidimos no programar todo en un solo archivo. Dividimos el proyecto en tres módulos principales: uno para generar los datos, uno para ordenarlos y otro para calcular la estadística. Al separar el código de esta forma, evitamos que las funciones se revuelvan. En la Fig. 1 se muestra cómo está estructurado el proyecto y cómo se pasa el arreglo de datos entre los diferentes archivos del programa.

```
generador-analizador-estadistico/  
├── .gitignore  
├── .gitattributes  
├── Makefile  
├── README.md  
├── bin/  
├── build/  
├── docs/  
│   ├── html/  
│   ├── Reporte_Practica2.pdf  
│   └── Doxyfile  
├── include/  
│   ├── estadistica.h  
│   ├── generacion.h  
│   └── ordenamiento.h  
└── src/  
    ├── estadistica.c  
    ├── generacion.c  
    ├── main.c  
    └── ordenamiento.c
```

Figura 1. Estructura del proyecto.

II-A. Diseño del Programa

El software se hizo separando el código en archivos fuente (.c) y sus cabeceras (.h). La estructura principal es la siguiente:

- `src/main.c`: Es el archivo principal. Aquí está el menú interactivo, la validación para que el programa no falle si el usuario mete letras en lugar de números, y la lectura de argumentos desde la consola usando `argc` y `argv`.
- `src/generacion.c` e `include/generacion.h`: Contienen las fórmulas matemáticas para generar los datos aleatorios según la distribución elegida.

El flujo del programa funciona así: primero, el módulo de generación llena un arreglo global de tipo `double`. Luego, pasamos a la parte de ordenamiento. Para no alterar los datos originales y poder hacer las estadísticas correctamente después, hicimos una copia del arreglo usando memoria dinámica (`malloc` y `memcpy`). Sobre esta copia medimos el tiempo de los algoritmos.

Para generar los números aleatorios, usamos la función `rand()` de C. Sin embargo, para evitar problemas matemáticos (como calcular el logaritmo de cero), tuvimos que transformar los valores enteros que da `rand()` a números decimales entre 0 y 1 usando esta fórmula obligatoria:

$$u = \frac{\text{rand}() + 1,0}{\text{RAND_MAX} + 2,0} \quad (1)$$

Usando esta variable u , programamos las tres distribuciones:

- **Distribución Uniforme** $U(\text{mín}, \text{máx})$: Escala los números directamente para que caigan entre el mínimo y máximo:

$$x = \text{mín} + (\text{máx} - \text{mín}) \cdot u \quad (2)$$

- **Distribución Normal** $\mathcal{N}(\mu, \sigma^2)$: Para generar la curva de campana, usamos el método de Box-Muller [1]. Toma dos valores uniformes aleatorios (u_1, u_2) y los transforma con la siguiente fórmula, donde μ es la media y σ la desviación estándar:

$$x = \mu + \sigma \cdot \left(\sqrt{-2,0 \cdot \ln(u_1)} \cdot \cos(2\pi \cdot u_2) \right) \quad (3)$$

- **Distribución de Laplace** Para esta distribución usamos el método de la Transformación Inversa [2]. Con la función de signo (`sgn`), calculamos cada dato usando la localización μ y la escala b :

$$x = \mu - b \cdot \text{sgn}(u - 0,5) \cdot \ln(1,0 - 2,0 \cdot |u - 0,5|) \quad (4)$$

II-B. Modos de Entrada de Datos

El programa le permite al usuario elegir cómo quiere definir los límites de sus datos de dos formas:

- **Por Rango**: El usuario escribe el valor mínimo y máximo. El programa calcula la media y desviación asumiendo que el rango equivale aproximadamente a $\pm 3\sigma$.
- **Por Parámetros**: El usuario escribe directamente la Media (μ) y la Varianza (σ^2). Con esto, el código calcula automáticamente cuáles deberían ser los límites de la gráfica.

II-C. Restricción Fuera de Rango (Recorte)

Al usar la distribución Normal o Laplace, a veces la fórmula matemática genera valores atípicos que, por sus colas probabilísticas, se salen de los límites físicos $[\text{mín}, \text{máx}]$ exigidos para la simulación.

Para solucionar esto, implementamos la técnica de **recorte (clipping)**. Si un valor generado sobrepasa el límite máximo, el programa lo restringe automáticamente e iguala su valor a `máx`; de la misma forma, si cae por debajo del mínimo, lo fuerza a ser igual a `mín`. Aunque esta estrategia exhibe un costo computacional inferior, es importante documentar que crea una acumulación artificial de muestras en las fronteras de la gráfica, lo cual puede alterar ligeramente el comportamiento de los momentos estadísticos de orden superior (como la varianza y curtosis de la muestra).

II-D. Uso de Semilla

Para poder comprobar que el programa funciona bien, usamos `srand(semilla)` en lugar de `time(NULL)`. Al meterle siempre el mismo número de semilla (ya sea desde el menú o desde los argumentos de la consola), el programa nos genera exactamente el mismo arreglo aleatorio, lo que nos permitió validar nuestros tiempos de manera determinista.

III. ALGORITMOS DE ORDENAMIENTO

III-A. Algoritmos Clásicos $\mathcal{O}(n^2)$ y Shell Sort

Los algoritmos básicos, como Burbuja, Selección, Inserción, Cocktail y Pares e Impares (Odd-Even), basan su funcionamiento en revisar elementos uno a uno e intercambiarlos si están al revés. Aunque son fáciles de programar, se vuelven extremadamente lentos cuando el tamaño del arreglo crece mucho, llegando a una complejidad de $\mathcal{O}(n^2)$. Para un total de $n = 100,000$ datos, estos métodos requerirían miles de millones de operaciones de comparación, haciendo inviable su ejecución en la práctica debido a los tiempos de espera en la terminal.

Por otro lado, Shell Sort representa una mejora muy importante sobre los métodos básicos. En lugar de comparar elementos que están pegados, introduce el concepto de “saltos” o espacios (gaps). Divide el arreglo original en subgrupos formados por datos separados a una distancia fija, aplicando Insertion Sort en cada grupo separado. Conforme el proceso avanza, la distancia del salto se reduce a la mitad hasta llegar a 1. Este truco reduce drásticamente el desorden del arreglo en las primeras pasadas, bajando la complejidad promedio a cerca de $\mathcal{O}(n^{1,5})$, lo que permite ordenar miles de datos de forma casi instantánea.

III-B. El Reto de los Decimales en Counting y Bucket Sort

El algoritmo Bucket Sort funciona creando contenedores o “cubetas” donde reparte los números de acuerdo con su valor para luego ordenar cada cubeta por separado. En nuestro programa, lo adaptamos para recibir variables continuas con decimales de tipo `double` mapeando de forma proporcional el valor mínimo y máximo del experimento hacia los índices enteros de un arreglo de listas ligadas. Este algoritmo resultó ser el rey de la velocidad cuando trabajamos con la Distribución Uniforme, ya que al estar los datos repartidos de manera pareja en todo el rango, cada cubeta recibe casi la misma cantidad de datos y el ordenamiento fluye de inmediato. Sin embargo, cuando se introducen datos con Distribución Normal o de Laplace, el rendimiento cae drásticamente porque estas distribuciones concentran casi todas las muestras alrededor del centro de la campana. Esto causa que una sola cubeta se sature con la gran mayoría de los datos del programa, obligando al algoritmo a ordenar un bloque gigante de forma lineal y anulando su ventaja de velocidad.

Para el algoritmo Counting Sort enfrentamos un reto todavía mayor, ya que este método requiere por definición trabajar exclusivamente con índices enteros positivos para contar frecuencias. Para solucionar esto y poder ordenar los números con punto decimal de la simulación, implementamos

un factor de escala multiplicando cada dato por 1000,0 para transformarlo en entero, y luego dividiendo entre el mismo factor al reconstruir el arreglo. Esto nos permitió aprovechar su velocidad lineal $\mathcal{O}(n + k)$, pero trajo consigo un costo matemático importante: al hacer el truncamiento a entero, se destruyen los decimales que van más allá del tercer dígito, lo que provoca una pérdida irreversible de precisión matemática en los datos originales del experimento. Además, si el rango entre el mínimo y el máximo de la prueba es muy grande, la diferencia entre enteros genera un rango tan inmenso que el programa se quedaría sin memoria RAM al intentar alojar el arreglo de frecuencias, razón por la cual tuvimos que meter un salvavidas en el código que cambia el proceso a Shell Sort automáticamente si se detecta este peligro.

IV. RESULTADOS

IV-A. Simulación de los 20 Escenarios

A continuación, en la Tabla I, se presentan los resultados de los 20 escenarios de simulación requeridos en la práctica, alternando las tres distribuciones para su análisis integral. Las últimas dos columnas documentan el aislamiento de los tiempos de cómputo exigidos por la rúbrica.

IV-B. Análisis de Complejidad

Para entender por qué algunos algoritmos tardaron más que otros en nuestras pruebas, comparamos su complejidad algorítmica de tiempo (cuántas operaciones hacen) y de memoria (cuánto espacio extra necesitan en la RAM). Esto se resume en la Tabla II usando la notación Big- $\mathcal{O}$.

Cuadro II
COMPLEJIDAD TEMPORAL Y ESPACIAL DE ALGORITMOS

Algoritmo	Mejor Caso	Promedio	Peor Caso	Espacial
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Odd-Even Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Bucket Sort	$\mathcal{O}(n + k)$	$\mathcal{O}(n + k)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n + k)$
Cocktail Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Shell Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^{1.5})$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Counting Sort	$\mathcal{O}(n + k)$	$\mathcal{O}(n + k)$	$\mathcal{O}(n + k)$	$\mathcal{O}(k)$

Durante la ejecución de las pruebas en la consola de CLion con la semilla global fijada en 23, nos topamos con cuellos de botella muy claros que demuestran la teoría de la complejidad Big- $\mathcal{O}$. Cuando trabajamos con tamaños pequeños de muestras ($n = 100$ o $n = 1000$), la potencia de procesamiento de la computadora hace que algoritmos básicos como Bubble Sort o Selection Sort terminen en 0,0000 segundos. Sin embargo, al brincar a escenarios medianos, el comportamiento cambió por completo: en el Caso 17 ($n = 10000$ con distribución Normal), el algoritmo clásico elevó el tiempo a unos notorios 0,1600 segundos. Si hubiéramos intentado correr Bubble Sort para el Caso 20 ($n = 50000$), el programa habría tardado minutos enteros haciendo comparaciones repetitivas, dando la impresión de que la terminal se había congelado en un ciclo infinito. En contraste, gracias a los saltos logarítmicos

de Shell Sort y la separación por cubetas de Bucket Sort, logramos ordenar los tamaños masivos de 30,000 y 50,000 datos en apenas milésimas de segundo (0,0130 s y 0,0020 s respectivamente), evidenciando que para arreglos gigantescos es obligatorio abandonar los métodos de ordenamiento elementales.

IV-C. Evidencias de Ejecución

A continuación, mostramos capturas de pantalla de la ejecución del programa. En la Fig. 2 se ve cómo el menú maneja los errores si el usuario presiona una tecla inválida, limpiando el búfer para evitar ciclos infinitos.

```
PS C:\Users\quime\Git\generador-analizador-estadistico> .\programa.exe Laplace 200 -50 10 1234
Semilla usada: 1234
Modo: rango | [-50.0000, 10.0000]
Distribución invalida: Laplace
PS C:\Users\quime\Git\generador-analizador-estadistico>
```

Figura 2. Menú principal con validación de entradas.

En la Fig. 3 se muestra la prueba usando la línea de comandos, pasándole la semilla y los parámetros directamente desde los argumentos `argc` y `argv`.

```
PS C:\Users\quime\Git\generador-analizador-estadistico> .\programa.exe Laplace 200 -50 10 1234
Semilla usada: 1234
Modo: rango | [-50.0000, 10.0000]
Datos generados: 200

--- ORDENAMIENTO ---
1. Bubble Sort
2. Odd-Even Sort
3. Selection Sort
4. Insertion Sort
5. Bucket Sort
6. Cocktail Sort
7. Shell Sort
8. Counting Sort
9. Volver al menu principal
Opcion: 5

Ordenando 200 datos...
Tiempo de ordenamiento: 0.000000 segundos

Calculando medidas estadísticas...

=====
RESULTADOS ESTADISTICOS (n = 200)
=====

--- Posicion central ---
Maximo      : 8.286453
Minimo      : -50.000000
Media aritmetica : -20.307596
Media geometrica : nan
Mediana     : -19.761685
```

Figura 3. Ejecución directa desde la terminal (línea de comandos).

V. CONCLUSIONES

Logramos desarrollar un programa estable en C que genera y ordena grandes arreglos de datos usando diferentes tipos de probabilidad. Cumplimos con el requisito de no usar una misma función para dos medidas estadísticas y pudimos comprobar cómo la semilla de verdad hace que las simulaciones sean reproducibles.

En conclusión, pudimos demostrar experimentalmente que el rendimiento de un algoritmo de ordenamiento no depende solo de la cantidad total de datos (n), sino también del comportamiento matemático de los valores generados. Para escenarios regidos por la Distribución Uniforme, los algoritmos basados en distribución lineal como Bucket Sort resultaron los ganadores indiscutibles, ya que la simulación esparce los

Cuadro I
RESULTADOS ESTADÍSTICOS Y TIEMPOS DE SIMULACIÓN PARA 20 CASOS

Caso	Distribución	n	Mín	Máx	Media	Mediana	Varianza	Tiempo Ord. (s)	Tiempo Est. (s)
1	Uniforme	100	-0.9930	0.9925	0.0901	0.1976	0.3177	0.0000	0.0810
2	Normal	1000	-5.0000	5.0000	-0.0021	-0.0170	2.7375	0.0000	0.0880
3	Laplace	1000	-10.0000	20.0000	5.2624	5.0265	23.4334	0.0010	0.0900
4	Uniforme	1000	-49.9161	-0.0153	-24.4314	-24.8131	208.3933	0.0030	0.0810
5	Normal	1000	-1.0000	1.0000	-0.0004	-0.0034	0.1095	0.0030	0.0850
6	Laplace	2000	10.0000	30.0000	20.0525	20.0053	4.6867	0.0020	0.1700
7	Uniforme	2000	-30.0000	30.0000	-0.0046	-0.0635	300.0000	0.0010	0.1650
8	Normal	2000	-50.0000	-10.0000	-30.0004	-30.0034	43.8004	0.0020	0.1640
9	Laplace	2000	-100.0000	150.0000	25.5249	25.0531	117.1672	0.0000	0.1700
10	Uniforme	2000	-1.0000	1.0000	-0.0002	-0.0021	0.3333	0.0080	0.1690
11	Normal	4000	-1.0000	1.0000	-0.0002	-0.0017	0.1095	0.0000	0.3230
12	Laplace	5000	-1.0000	1.0000	-0.0048	-0.0053	0.0469	0.0010	0.4130
13	Uniforme	6000	-1.0000	1.0000	-0.0001	-0.0007	0.3333	0.0460	0.4900
14	Normal	7000	-1.0000	1.0000	-0.0001	-0.0010	0.1095	0.0380	0.5690
15	Laplace	8000	-1.0000	1.0000	-0.0030	-0.0033	0.0469	0.0570	0.6470
16	Uniforme	9000	-1.0000	1.0000	-0.0000	-0.0005	0.3333	0.0760	0.7250
17	Normal	10000	-1.0000	1.0000	-0.0001	-0.0007	0.1095	0.1600	0.8040
18	Laplace	20000	-1.0000	1.0000	-0.0024	-0.0016	0.1024	0.0030	0.1630
19	Uniforme	30000	-1.0000	1.0000	-0.0000	-0.0002	0.3333	0.0130	2.4410
20	Normal	50000	-1.0000	1.0000	-0.0000	-0.0003	0.1095	0.0020	4.0320

números de manera simétrica por todo el espacio, impidiendo la saturación de los contenedores de memoria.

Por el contrario, para las distribuciones de tipo Normal y Laplace, los datos tienden a agruparse fuertemente alrededor de la media debido a sus formas de campana y picos exponenciales. Esto arruinó por completo la estrategia de cubetas de Bucket Sort al amontonar casi todas las muestras en un único nodo central. Bajo estas condiciones probabilísticas, Shell Sort demostró ser el método más eficiente, seguro y robustez de todo el proyecto, acomodando 50,000 datos continuos de forma casi inmediata sin sacrificar precisión decimal ni poner en riesgo la memoria RAM de la computadora.

En resumen, nuestras pruebas demostraron que no existe un "mejor algoritmo." absoluto. La rapidez de los métodos más avanzados depende casi por completo del tipo de distribución que elija el usuario y de cómo se comportan esos datos matemáticamente.

REFERENCIAS

- [1] Alexander and Alexander, "Box muller transform: Simple definition," *Statistics How To*, Mar. 2023. [En línea]. Disponible en: <https://www.statisticshowto.com/box-muller-transform-simple-definition/>
- [2] L. F. Chaparro and A. Akan, "The Laplace transform," en *Elsevier eBooks*, pp. 167–240, 2018. [En línea]. Disponible en: <https://doi.org/10.1016/b978-0-12-814204-2.00013-2>
- [3] L. Devroye, *Non-Uniform Random Variate Generation*, 1ra ed. Nueva York, EE. UU.: Springer-Verlag, 1986, pp. 27–35.